

Aho-Corasick Algorithm for Multiple String Matching

Introduction

The Aho-Corasick algorithm is a fast algorithm for string-matching. It is primarily used to locate numerous patterns in a text in one pass. This is unlike the string matching methods that typically look for just one pattern at a time. It was devised by Alfred V. Aho and Margaret J. Corasick in 1975, and is still employed in many current systems requiring rapid pattern detection.

Multiple string matching is significant because in lots of actual-world applications we may need to look for not one word or one pattern; we need to look for several ones. An intrusion detection system can, for example, be required to perform a search of network packets for thousands of known signatures of attacks. A virus scanner can have to contrast files with a huge database of dubious strings. A DNA or protein sequence can be searched with a bioinformatics application for numerous known biological patterns. For these instances, the use of one-pattern algorithm repeatedly would be inefficient.

Aho-Corasick is solving this problem by constructing one integrated state machine for all the patterns. Once this structure is constructed, the text to be processed can be read one character at a time. The algorithm has the ability to output all patterns that match the text at one scan. This makes it useful in many applications such as large-scale text processing, network security, spam filtering, virus scanning, and biological sequence analysis. Aho-Corasick is introduced in several papers within the sources made available, in particular in the context of intrusion detection systems, hardware acceleration and multicore processing.

What is the algorithm supposed to do? What does the algorithm do?

The Aho-Corasick algorithm consists of two major steps. Preprocessing is the first step. The algorithm creates a trie in this stage from the set of the patterns. A trie is a tree data structure with common prefixes sharing. As an example, instead of storing the patterns “he”, “she”, “his”, and “hers” separately, the algorithm only stores two of the four patterns. Rather, it keeps the common components in the same structure.

Once the trie has been constructed the algorithm adds failure links. These connections enable the algorithm to keep searching if the next character differs from what's on the current path. The algorithm does not return to the initial state in the text, but rather proceeds to another helpful state. This is one of the important components of Aho-Corasick where there is no repetition of work.

Searching is the second stage. During this step, the algorithm goes through the text, looking at it from left to right. At each character, the state machine moves from one state to another. When the algorithm encounters an output state, this indicates that a

pattern(s) has been detected. There can be overlaps in some matchings and the algorithm still detects them. For instance in the word “she”, if the pattern set contains the “she” pattern as well as the “he” pattern, both “she” and “he” can be detected in the word.

A simplified pseudocode can be shown as follows:

Build trie from all patterns

Add failure links

state = root

for each character in text:

 while no transition exists:

 state = failure link

 move to next state

 if state has output:

 report matched pattern

This pseudocode shows the main idea without going into low-level implementation details.

Algorithmic Complexity

The search time of the Aho-Corasick algorithm is linear in the length of the text being searched, making it an efficient algorithm. Preprocessing includes the ability to skip the need to search the text for each pattern after preprocessing. It reads the input stream only once and tells at each point when it finds a match.

The general complexity is $O(n + m + z)$, with n being the length of the input text, m being the total length of all of the patterns, and z being the number of matches found. The preprocessing stage is very dependent on the size of the pattern set, and the search stage is primarily dependent on the input length and the number of matches.

This is very useful in systems that have large pattern databases. The runtime of string matching algorithms used by intrusion detection systems should not be proportional to the size of the rule base as the number of packets they scan increases, explains Tuck, et al. This is appropriate for Aho-Corasick as it can guarantee worst case behavior.

But the algorithm has one drawback. The state machine may consume a lot of RAM, particularly if it is realized as a deterministic finite automaton. Pao, Lin and Liu provide an explanation that Aho-Corasick combines all the strings into a single automaton, and keeping all the transition rules in memory can be costly. For instance, if there are a lot of strings and each of the states has a lot of transitions, then the transition table can be too large to be used in an embedded or hardware system.

Practical Uses

Ahocorasick is one of the most significant ones in network intrusion detection system. The systems use a dictionary of known attack signatures to check network packets. One packet stream can have to be compared to thousands of rules. Aho-Corasick is useful for this type of real-time packet inspection due to the fact that many signatures can be searched in a single pass.

Tuck et al. have talked about this issue with respect to the popular intrusion detection system Snort. They explain that string matching can take a large part of the total execution time in such systems. This is an illustration of the importance of designing an algorithm that is both quick and uses little memory in the context of security applications. They are also interested in minimizing the memory consumption of Aho-Corasick without sacrificing its worst case performance.

Another real application is hardware acceleration. In some systems, the speed of processing the data is extremely important, particularly in routers, firewalls and network monitoring devices. Chen and Wang describe how to implement an Aho-Corasick in hardware and, since exact string matching is frequently performed first to identify suspicious data, they explain the reasoning behind that. They also explain how to use multiple transitions between characters (more than 1 character can be analyzed simultaneously to boost throughput).

The Aho-Corasick is also of use in bioinformatics. There can be a huge number of sequences in biological databases, and many patterns inside DNA or protein sequences might need to be searched. Arudchutha, Nishanthi and Ragel write that multiple string matching plays an important role in biological sequence analysis and that multicore processors can be employed to enhance the performance of the Aho-Corasick implementation.

Comparison with KMP Algorithm

Aho-Corasick and Knuth-Morris-Pratt algorithm (KMP) are useful to compare. KMP is mainly used for single pattern matching. It is efficient for looking for the one pattern in a text. It preprocesses the pattern and then does a linear search of the text.

But if there are lots of patterns, KMP is not suitable. Suppose there are 1000 patterns, one of the simple solutions would be to execute KMP 1000 times. This means that the text entered into the machine might have to be scanned numerous times. Aho-Corasick's solution is to merge all patterns into a single state machine. All the patterns are detected at once after scanning the text once.

Pao, Lin, and Liu also note that typical hardware implementations using KMP would need to have one matching unit for each string. This may prove to be costly if the signature set is large. Aho-Corasick, by comparison, has a signature set in a single automaton and is better for large sets of signatures.

Hence, KMP is a better choice when there is one pattern and Aho-Corasick is better when there are many patterns. This makes Aho-Corasick more useful for intrusion detection and large dictionary matching and malware scanning.

Advantages and Limitations

The best thing about Aho-Corasick is that it can search for multiple patterns. It performs reliably, and doesn't re-scan the same input over and over again. This is helpful for systems that require performance—particularly in real-time systems.

Another benefit is it could discover overlapping patterns. This is relevant as it is possible to have patterns within patterns in the case of real text or network data. The algorithm is also adaptable to hardware, multicore CPUs and parallel systems.

However, the main limitation is memory usage. For Classical Aho-Corasick, a large transition table can be required. A problem for FPGA, ASIC, router or embedded. This led to various collapsed, pipelined and parallel implementations of the algorithm. Some examples include bitmap and path compression to minimize memory consumption (Tuck et al.) or a pipelined implementation to minimize memory cost while preserving a high speed (Pao et al.).

The other restriction is that a preprocessing step is required prior to searching. In some cases, the state machine might need to be rebuilt or updated if the pattern set changes frequently. This can be cumbersome in systems with frequently shifting rules.

Conclusion

The Aho-Corasick algorithm is a powerful multiple string matching algorithm. It is especially useful when a system must search for many patterns in a large input text. The algorithm builds a trie and failure links from the pattern set, then scans the input in one pass. Its linear search behavior makes it practical for high-performance systems.

Compared with KMP, Aho-Corasick is more suitable for multiple-pattern matching. KMP is strong for one pattern, but Aho-Corasick is better when hundreds or thousands of

patterns must be searched together. This is why Aho-Corasick is widely used in intrusion detection systems, virus scanners, spam filters, text processing, and bioinformatics.

The main weakness of the algorithm is memory usage, especially in hardware implementations. However, the sources show that many improved versions exist, including compressed, pipelined, multicore, and multicharacter implementations. These improvements make Aho-Corasick still relevant today, even though the original algorithm was introduced many years ago.

References

Arudchutha, S., Nishanthi, T., & Ragel, R. G. (2014). String Matching with Multicore CPUs: Performing Better with the Aho-Corasick Algorithm.

Chen, C.-C., & Wang, S.-D. (2013). An Efficient Multicharacter Transition String-Matching Engine Based on the Aho-Corasick Algorithm. *ACM Transactions on Architecture and Code Optimization*, 10(4).

Pao, D., Lin, W., & Liu, B. (2010). A Memory-Efficient Pipelined Implementation of the Aho-Corasick String-Matching Algorithm. *ACM Transactions on Architecture and Code Optimization*, 7(2).

Tuck, N., Sherwood, T., Calder, B., & Varghese, G. (2004). Deterministic Memory-Efficient String Matching Algorithms for Intrusion Detection. *IEEE INFOCOM*.